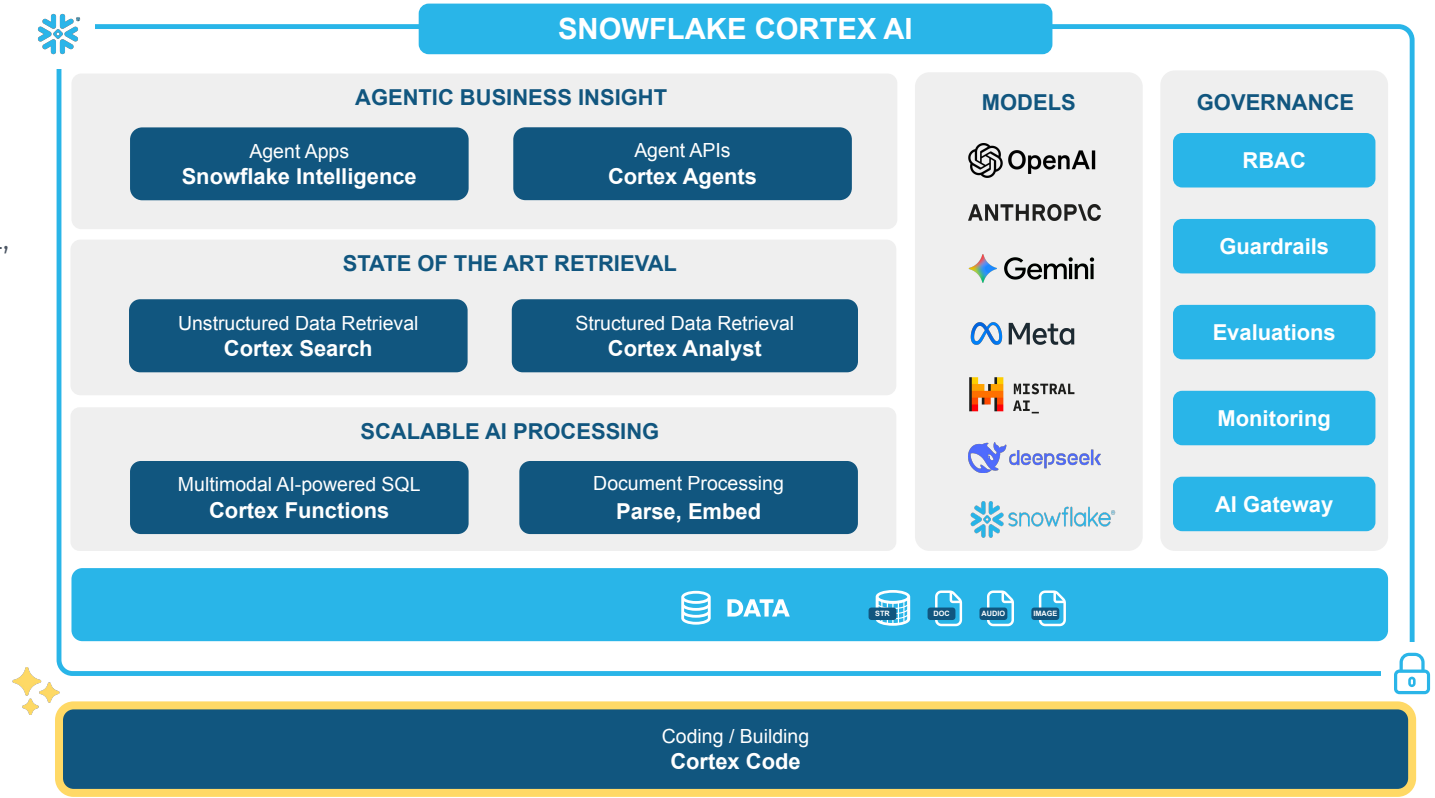
Snowflake Enablement: Cortex Code

April 2026

Native AI & Agents Inside Snowflake

Cortex Code

AI coding agent that generates and iterates on Snowflake-native code (SQL, Python, Streamlit, etc.) grounded in your data and environment.



Cortex Code: AI Coding Agent for Data Teams

THE CURRENT STATE

Traditional Dev Friction

FRAGMENTED TOOLCHAIN
Workflows spanning dbt, Airflow, SQL, and Notebooks → constant context switching.

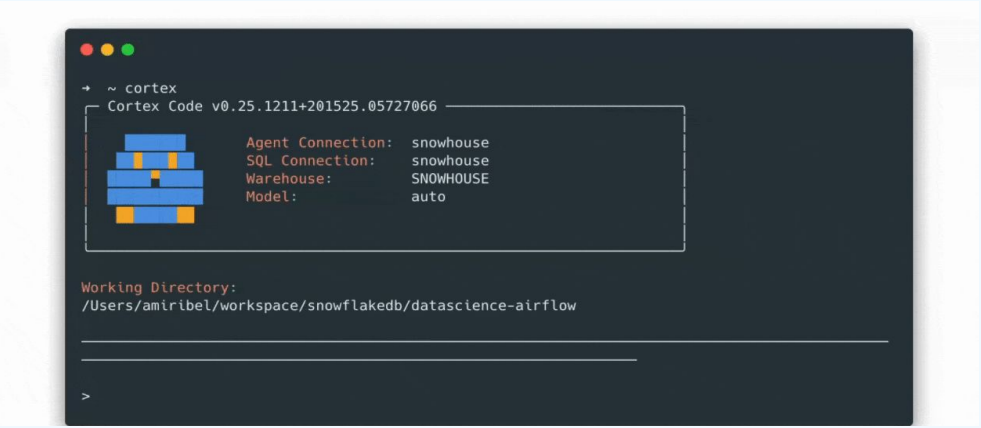
SLOW ITERATION
Manual refactoring and tribal knowledge slow debugging and deployments.

GOVERNANCE FRICTION
Access controls and policies bottleneck rapid prototyping.

THE SOLUTION

AI-Powered Development

From natural language to production-ready code across your entire data stack.
CLI & in Snowsight.



The Right Tool for Data Teams

An AI Coding Agent built to get you to production faster

UNDERSTANDS SNOWFLAKE

A built-in expert on Snowflake, it has up-to-the-minute knowledge of your specific data catalog: databases, schemas, semantic models and more.

INTELLIGENCE

CONTEXT AWARE

The agent knows where you are in the UI, what data you're working with, and retains your conversational history as you navigate.

RELEVANCE

AUTOMATES COMPLEX TASKS

Integrates natively with your stack. dbt, Git, SQL, Python, and Streamlit. No changes to how you work.

INTEGRATION

SECURE BY DESIGN

Cortex Code uses Snowflake RBAC, ensuring it only sees and acts on the data and objects you have access to

GOVERNANCE

Accelerate Your Data and AI Workflows

CORTEX CODE

DATA DISCOVERY

Search the catalog for data with semantic understanding

ADMIN TASK

Manage users, permissions, costs and governance

DATA ENGINEERING

Build & optimize pipelines with dbt and AirFlow

AI & AGENTS

Build, evaluate, optimize, debug agents and semantic views

ML & DATA SCIENCE

Auto-generate ML pipelines and feature engineering

ADVANCED ANALYTICS

Complex Analysis with Python and SQL generation

BI & APPS

Streamlit apps, dashboards & data applications

CODE MIGRATION & REFACTORING

Refactor and convert code from other platforms

UNDERSTANDS SNOWFLAKE & DATA

AUTOMATES COMPLEX TASKS

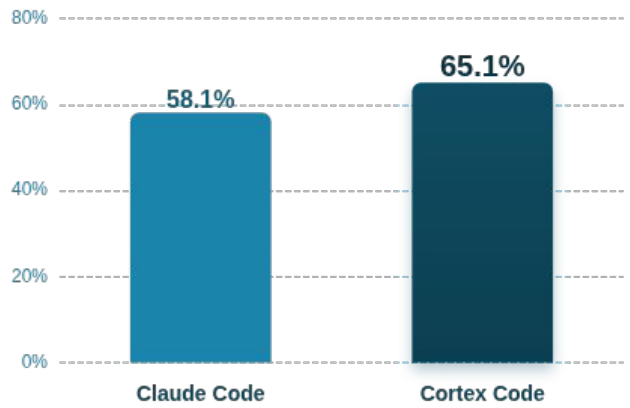
CONTEXT AWARE CLI & UI

SECURE BY DESIGN

Cortex Code Is the World's Best Data Coding Agent

Pass Rates on ADE-Bench

Higher is better (43 total tasks)



Model Version: Opus 4.6

BENCHMARK RESULTS

Cortex Code achieves comparable rates but demonstrates superior execution efficiency, with a **15.7% higher efficiency score** compared to Claude Code.

TOOL CALLS

854 ↓ 48%

vs 1,657 (Claude)

AVG STEPS

19.9 ↓ ~50%

vs 38.5 (Claude)

OPERATIONS

File Reads	2x Fewer
Bash Cmds	4x Fewer

Why Cortex Code is More Efficient

TARGETED VS. EXHAUSTED

Cortex performs targeted exploration rather than exhaustive scanning, significantly reducing IO operations.

SQL-NATIVE APPROACH

Utilizes **snowflake_sql_execute** for native execution, avoiding heavy reliance on bash tools common in Claude's workflow.

* Analytics and Data Engineering Benchmark by DBT

Snow-on-Snow: 10x Faster Root Cause Analysis for CX

10X

Faster Resolution

BEFORE
1 Hour to Days

AFTER
15 mins to 2 hrs

JIRA QUALITY
"Help us debug" → "Confirm this fix"

ENG EFFORT
Reduced from days to minutes

Context | Getting the right context together

Semantic Views
GET_ENVOY_LOGS, GET_JOBS for correct query results

Tool Integration
MCP connections to 10+ systems (Snowflake, SFDC, JIRA, GitHub, Glean, Confluence, etc.)

Intelligence | Data agent with encoded investigation patterns

Skills & Runbook
Reusable prompts encoding investigation patterns ("How do I debug throttling?")

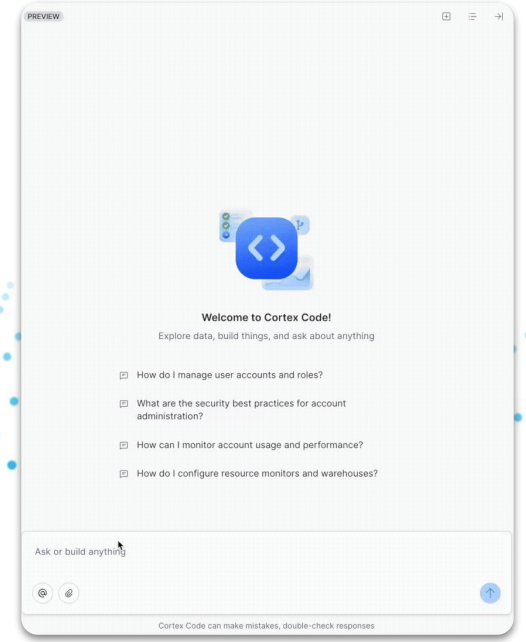
Planning & Reasoning
Dynamic investigation paths & anomaly detection

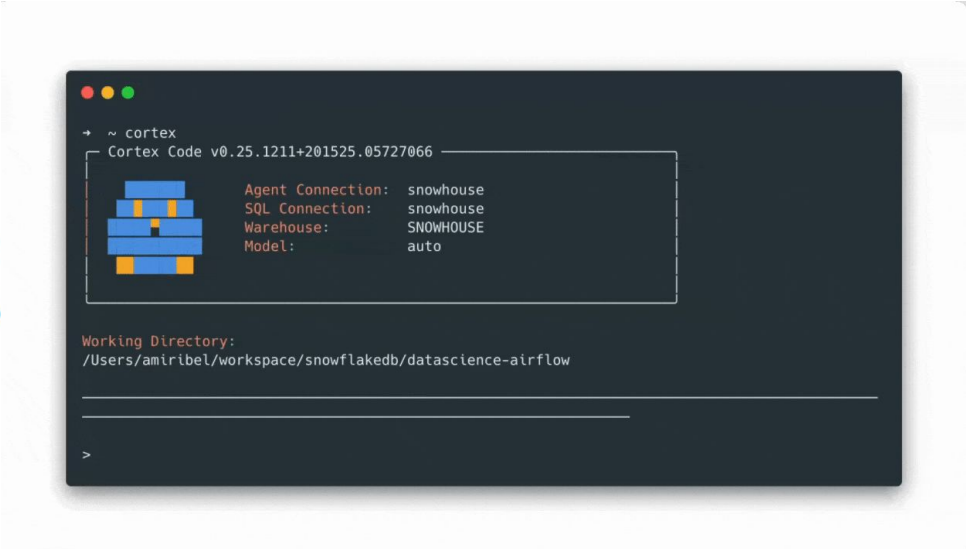
Interface | Surfaced to Engineers in Cortex Code

Profiles
Packaged configurations (skills + tools + settings) stored centrally

No Setup Required
Any engineer gets the same toolset instantly available

Where to Access Cortex Code





Snowsight

CLI

BUILDING WITH BEST PRACTICES

GOVERNANCE

Governance - Key Pillars

ACCESS MANAGEMENT

Fine-grained access control to ensure no data, agents, or other artifacts fall into the wrong hands.

```
> What tables are available in the CUSTOMER_DATA db

* Let me search for tables in the CUSTOMER_DATA database.

× SNOWFLAKE_SQL_EXECUTE List all tables in CUSTOMER_DATA database
  └ Error executing SQL: SQL compilation error:

✓ SNOWFLAKE_SQL_EXECUTE Search for CUSTOMER_DATA database
  0 row(s) affected.

* The CUSTOMER_DATA database doesn't exist or your current role
  doesn't have access to it.

  Let me check what databases are available:
```

PERMISSIONING MODES

Confirm, Plan, and Bypass modes allow for precise control of local and remote operations.

```
> Create a new folder in my root directory called test

✓ BASH ls ~/test 2>/dev/null; echo "EXIT:$?"
  └ EXIT:1

Execute Command [HIGH]
 mkdir ~/test

  ○ 1. Yes
  ○ 2. Allow this specific command in the future
  ○ 3. Allow this specific command for this session
  ○ 4. Always allow using 'mkdir'
  ○ 5. Allow using 'mkdir' for this session
❯ ○ 6. No (tab to add instructions)
↑↓ enter 1-6 esc
```

MODEL RBAC

Control which models are available account wide or to individual Cortex Code Users. Leverage Auto mode to pick best model available.

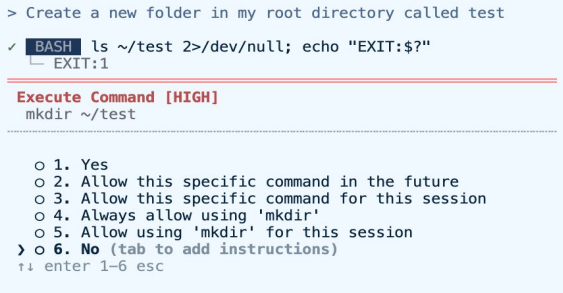
```
Select a model

❯ ○ Auto - Picks best available model for your account/region (current)
  ○ Claude Opus 4.6
  ○ Claude Opus 4.5
  ○ Claude Sonnet 4.6
  ○ Claude Sonnet 4.5
  ○ Claude Sonnet 4.0
  ○ GPT-5.2
↑↓ enter esc
```

Permission Modes Deep Dive

Confirm Mode (default)

Prompts the user before file edits, bash commands, etc. All proposed operations will classify risk level to user for approval.

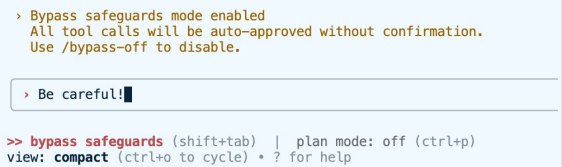


Plan Mode (/plan)

Agent polls user for additional detail and proposes a plan but doesn't execute any action until approved.

⚠️ Bypass Mode (/bypass)

Auto-approves all tool without polling user (not recommended - can be disabled via managed settings)



Model RBAC Deep Dive

Model Governance

Account-Level Model Allowlist

An ACCOUNTADMIN sets the **`CORTEX_MODELS_ALLOWLIST`** parameter to explicitly permit or block models account-wide. Setting it to 'None' blocks ALL model access unless explicitly re-enabled.

Per-Model RBAC (Fine-Grained Control):

Each Cortex model has its own application role in the **`SNOWFLAKE.MODELS`** schema (e.g., **`CORTEX-MODEL-ROLE-CLAUDE-SONNET-4-5`**). Grant or revoke access per role for explicit, auditable control.

Model Selection in Cortex Code

/model

In-session command to select from available models. Model selection applies only to current session.

CORTEX_AGENT_MODEL

Environment variable to set for model selection. Can also be included in settings.json like below.

{"env": {"CORTEX_AGENT_MODEL": "claude-sonnet-4-5"}}

cortex -- model

CLI Flag to specify model to be used in a specific session

Governance - Configuration Paths

Settings.json

Multiple layers of enforceable settings at the account or individual level to ensure company wide compliance and individual optimizations

```
{
    "version": "1.0",
    "permissions": {
      "onlyAllow": ["account(mycompany-prod)", "account(mycompany-staging)"],
      "deny": ["mcp__*"],
      "defaultMode": "allow",
      "dangerouslyAllowAll": false
    },
    "settings": {
      "forceNoHistoryMode": false,
      "forceSandboxEnabled": true,
      "forceSandboxMode": "regular"
    },
    "required": {
      "minimumVersion": "1.0.0"
    },
    "defaults": {
      "connectionName": "prod",
      "profileName": "corporate",
      "theme": "dark"
    },
    "ui": {
      "showManagedBanner": true,
      "bannerText": "[Secure] Managed by Corporate IT",
      "hideDangerousOptions": true
    }
  }
```

Hooks.json

hooks.json allows for customizable controls for allowing or preventing granular operations

Block destructive SQL (DROP, DELETE, TRUNCATE):

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "snowflake_sql_execute",
        "hooks": [
          { "type": "command", "command": "/path/to/block-destructive-sql.sh", "timeout": 10 }
        ]
      }
    ]
  }
}
```

Audit all Bash commands:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command", "command": "/path/to/audit-logger.sh", "timeout": 10 }
        ]
      }
    ]
  }
}
```

Block all file writes outside a project directory:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "write|edit",
        "hooks": [
          { "type": "command", "command": "/path/to/restrict-writes.sh", "timeout": 10 }
        ]
      }
    ]
  }
}
```

Connections.toml

Define connection profiles to various snowflake environments and configurations

```
> What connections do I have available

✓ BASH cortex connections list
  ├─ {
  ├─ "active_connection": "EBOTWICK_DEMO",
  └─ ... (23 more lines)
```

Connection	Account	User	Role
EBOTWICK_DEMO …	sfsenorthamerica-demo-…	EBOT…	ACCOUNTADM…
DEMO_ANALYST	sfsenorthamerica-demo-…	EBOT…	DATA_ANALY…
snowhouse	SFCOGSOPS-SNOWHOUSE_AW…	EBOT…	ENGINEER
JAN26_TRIAL	EHDRLVK-BAB26722	EBOT…	ACCOUNTADM…

Currently using **EBOTWICK_DEMO.** Switch with `cortex connections set <name>` or `/connection <name>`.

Governance - Best Practices

Enable Plan Mode - `/plan` - review actions before execution

Use Scoped Roles - Avoid using overly-permissioned roles with Cortex Code

Disable Bypass Mode - Managed settings: `"dangerouslyAllowAll": false`

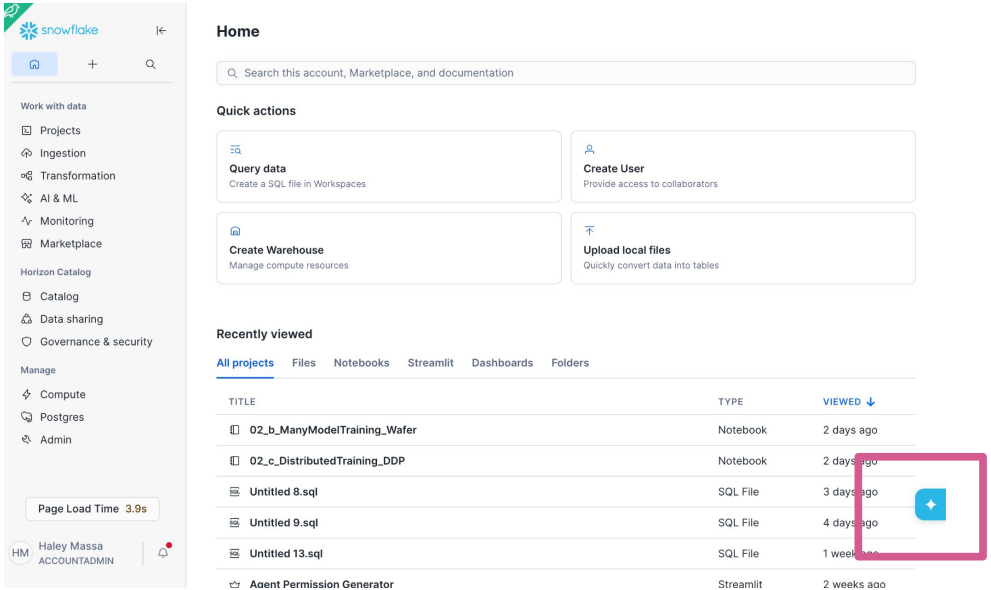
Deploy Hooks - Pre-execution hooks for production-specific policies

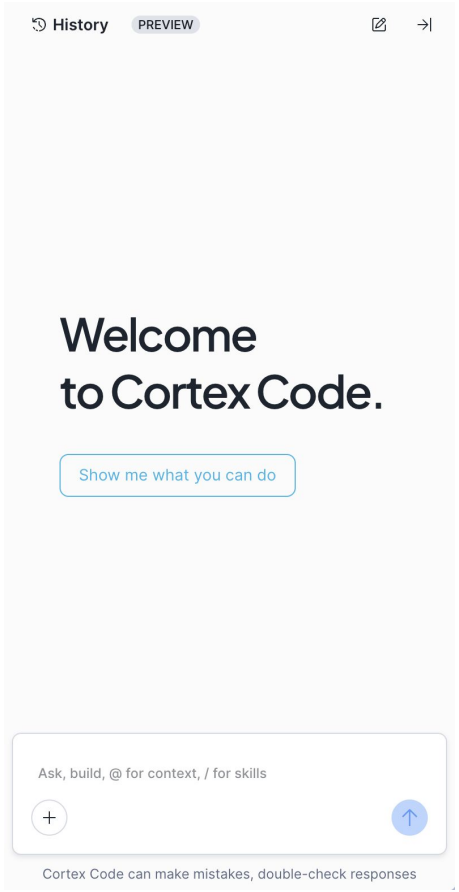
Force Sandbox - Via managed settings

Monitor Usage - Review `SNOWFLAKE.ACCOUNT_USAGE.CORTEX_CODE_CLI_USAGE_HISTORY` regularly

CREATING COCO CLI CONNECTION

Validate Cortex Code in Snowsight





Access Cortex Code via Snowsight by clicking on the blue star in the bottom right corner in the UI

Installing the CoCo CLI on Mac

INSTALL VIA TERMINAL

```
curl -LsS https://ai.snowflake.com/static/cc-scripts/install.sh | sh
```

Architectures

- x64 – Supported.
- ARM64 – Supported

VERIFY INSTALL

```
cortex --version
```

During Installation

- When prompted to add cortex to your PATH, respond with y.
- If the cortex command is not recognized after installation:
- Close and reopen your terminal, or
- Restart your machine to ensure PATH changes take effect.

Installing the CoCo CLI on Mac

CONFIGURE SNOWFLAKE CONNECTIONS

- Cortex Code uses the same connection files as SnowCLI, under your home directory. On Macs that's typically:
- ~/.snowflake/config.toml (or connections.toml if you already use that).
- Minimal example (adapt to your accounts/roles):

```
default_connection_name = "DEMO"  # NAME FOR DEFAULT SNOWFLAKE CONNECTION
[connections.DEMO]  # NAME FOR SNOWFLAKE CONNECTION (SAME AS ABOVE IF DEFAULT)
account   = "<YOUR_DEMO_ACCOUNT>"      # e.g. ACCOUNT IDENTIFIER
user      = "<YOUR_DEMO_USERNAME>"     # e.g. AUTHENTICATION VARIABLE
password  = "<YOUR_DEMO_PAT>"
role      = "<YOUR_DEMO_ROLE>"         # e.g. NEED A ROLE FOR CONNECTION
warehouse = "<YOUR_DEMO_WAREHOUSE>"
```

LAUNCH CORTEX CODE CLI USING YOUR SQL CONNECTION

```
cortex -c DEMO # connect to connection name
```

Installing the CoCo CLI on Windows

INSTALL VIA POWERSHELL

```
irm https://ai.snowflake.com/static/cc-scripts/install.ps1 | iex
```

Architectures

- x64 – Supported.
- ARM64 – Support is in progress. Current builds may rely on experimental runtime support and are not yet recommended for production use.

VERIFY INSTALL

```
cortex --version
```

During Installation

- When prompted to add cortex to your PATH, respond with y.
- If the cortex command is not recognized after installation:
  - Close and reopen your terminal, or
  - Restart your machine to ensure PATH changes take effect.

Installing the CoCo CLI on Windows

CONFIGURE SNOWFLAKE CONNECTIONS

- Cortex Code uses the same connection files as SnowCLI, under your home directory. On Windows that's typically:
- %USERPROFILE%.snowflake\config.toml (or connections.toml if you already use that).
- Minimal example (adapt to your accounts/roles):

```
default_connection_name = "DEMO"  # NAME FOR DEFAULT SNOWFLAKE CONNECTION
[connections.DEMO]  # NAME FOR SNOWFLAKE CONNECTION (SAME AS ABOVE IF DEFAULT)
account   = "<YOUR_DEMO_ACCOUNT>"      # e.g. ACCOUNT IDENTIFIER
user      = "<YOUR_DEMO_USERNAME>"     # e.g. AUTHENTICATION VARIABLE
password  = "<YOUR_DEMO_PAT>"
role      = "<YOUR_DEMO_ROLE>"         # e.g. NEED A ROLE FOR CONNECTION
warehouse = "<YOUR_DEMO_WAREHOUSE>"
```

LAUNCH CORTEX CODE CLI USING YOUR SQL CONNECTION

```
cortex -c DEMO # connect to connection name
```

Creating a Snowflake Connections Config

ON MAC

Run this code in your terminal to create the connections config file

```
mkdir -p ~/.snowflake
touch ~/.snowflake/config.toml
chmod 600 ~/.snowflake/config.toml
```

Then run this command to open the file to edit and add in config

```
open -e ~/.snowflake/config.toml
```

ON WINDOWS

Run this code in powershell to create the connection config file

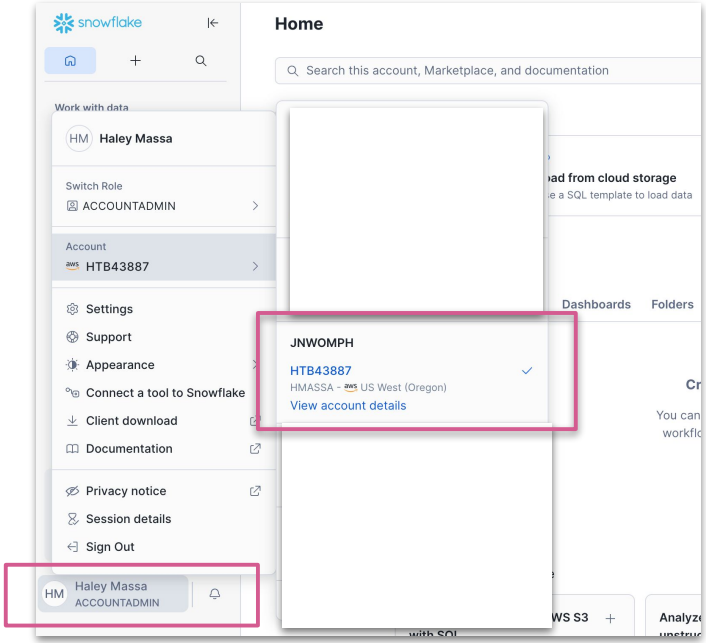
```
mkdir $env:USERPROFILE\.snowflake -Force
New-Item -ItemType File -Path $env:USERPROFILE\.snowflake\config.toml
-Force
```

Then run this command to open the file to edit and add in config

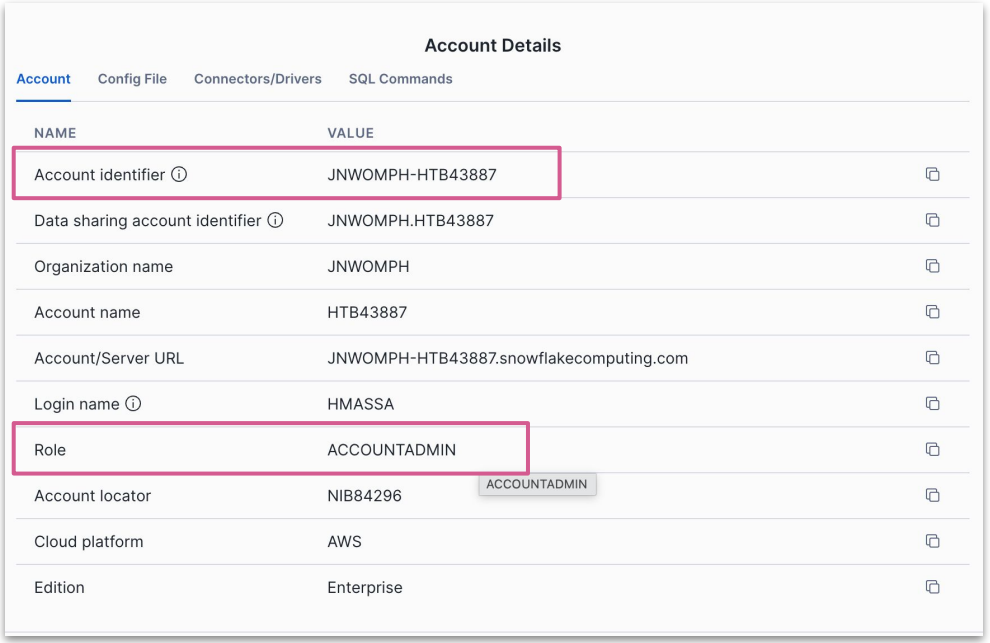
```
notepad $env:USERPROFILE\.snowflake\config.toml
```

Filling out the Default Connection

LOG IN AND FIND SETTING INFORMATION



CLICK ON CONNECTION AND FIND DEFAULTS



Ways to Authenticate Connections

CREATE AND AUTHENTICATE A CONNECTION TO SNOWFLAKE

There are many ways to create a connection to Snowflake. These are included into your connection configuration as credentials to identify you as logging into your connection. Here are some examples of environment variables used in Snowflake credentials:

- `SNOWFLAKE_ACCOUNT`
- `SNOWFLAKE_USER`
- `SNOWFLAKE_PASSWORD`
- `SNOWFLAKE_TOKEN`
- `SNOWFLAKE_TOKEN_FILE_PATH`
- `SNOWFLAKE_OAUTH_CLIENT_ID`
- `. . .`

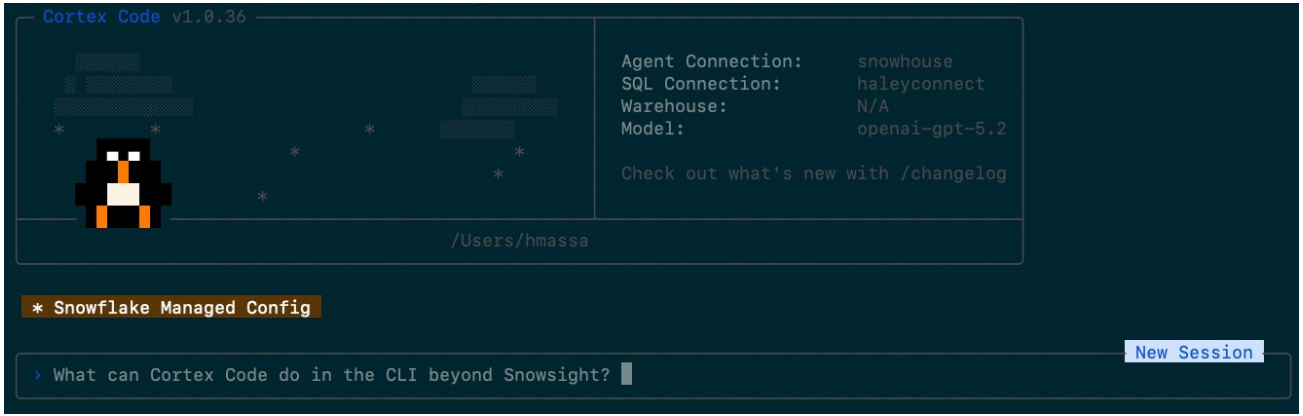
Find more examples and walk through creating your connection parameter in the Manage Snowflake connections guide.

Launching Cortex Code

LAUNCH CORTEX CODE CLI USING YOUR SQL CONNECTION

```
cortex -c DEMO # connect to connection name
```

- After connecting to cortex code (using the connect -c command and your connection name (in this example, DEMO), you will see the Cortex Code CLI interface
- From here is where we will cover more advanced usages of Cortex Code in our session, but we can test the connection first by asking a simple question:



GETTING STARTED WITH CORTEX CODE

How Cortex Code Works

Step 1: Install

Install the cortex CLI

Step 2: Connect

Connect to your Snowflake account (SSO, key-pair, or password)

Step 3: Ask

Start giving tasks in natural language

Key Interaction Patterns

#DB.SCHEMA.TABLE — auto-injects table metadata and sample rows into your prompt

/sql SELECT ... — run SQL inline and see results instantly

$skill-name — activate a specialized skill (e.g., $cost-intelligence)

@path/to/file — include file context in your conversation

Cortex Code CLI

Contains bundled tools and skills and possibility to extend further

Cortex Code CLI contains **tools** and bundled **skills** specific to Snowflake capabilities. Is as well **extendable** via **custom skills** and **MCP** servers.

Tools:

- Plan Mode
- SQL Execution
- Snowflake Documentation Access
- Local files access
- Terminal / Shell commands
- Web Search
- Browser
- Model Context Protocol (MCP)
- git Support
- dbt Support
- Skills (bundled and custom)
- Agents.md
- Hooks

Skills & Plugins (bundled):

Migration & Assessment

- SnowConvert Assessment

Data Engineering & Pipelines

- dbt on Snowflake
- Dynamic Tables
- Openflow

Security, Governance & Compliance

- Data Governance
- Data Policy
- Data Classification
- Cost Management

AI & Machine Learning

- Machine Learning
- Agent Optimization
- Semantic Views

Applications & Visualization

- Build React App
- Streamlit in Snowflake
- Deploy to SPCS
- Snowflake Postgres

Platform Operations

- Skill Development
- Cortex Code Guide

Essential Shortcuts & Commands

KEYBOARD SHORTCUTS

Shortcut	Action
Enter	Submit message
Ctrl+J	Insert newline
Ctrl+C	Cancel/Exit
Shift+Tab	Cycle modes (Confirm/Plan/Bypass)
Ctrl+P	Toggle Plan mode
Ctrl+D	Open full screen task view
Ctrl+B	Background bash process
Ctrl+O	Cycle display mode
Ctrl+V	Paste (supports images)

CONTEXT SHORTCUTS

Shortcut	Action	Example
@	File reference	@src/app.py
@file$10-50	Specific lines	@config.yaml$1-20
$	Invoke skill	$semantic-view-optimization
#	Snowflake table	#DB.SCHEMA.TABLE
!	Run bash command	!git status
/	Slash commands	/help, /sql, /plan
?	Quick help	?

CORTEX CODE WORKSHOP

Build A Snow Day Learning Fundamentals

Cortex Code Fundamentals

- [ ] Understand what Cortex Code is and how it differs from traditional development
- [ ] Access Cortex Code via Snowsight (UI)
- [ ] Configure a connection to run Cortex Code locally (CLI)
- [ ] See how Cortex Code understands your Snowflake environment (schema discovery)
- [ ] Execute SQL and workflows as yourself (your identity, your permissions)

Building with Cortex Code

- [ ] Build production-ready data pipelines in Snowflake
- [ ] Connect to external data/files and bring context into Snowflake
- [ ] Use bundled skills ($dynamic-tables, $semantic-view, etc)
- [ ] Create your own reusable skills for your team's workloads

Demo One: Pipeline Builder

From scattered ERP sources to a governed, chained Dynamic Table pipeline

The Story

Your company runs two ERP systems: SAP for manufacturing in Germany and Oracle for operations in the US. Both generate AP invoices, but with completely different column names and conventions.

Finance needs a single, consolidated view of all payables for reporting. Today this is done with manual SQL scripts that break every time a column changes.

You'll use Cortex Code to build a declarative pipeline using Dynamic Tables — Bronze to Silver (consolidation) to Gold (aggregation) — with automatic refresh and no orchestration code.

What You'll Learn

- Ground CoCo on live schema context
- Generate consolidation DDL from cross-source metadata
- Generate downstream aggregation DDL + verification queries
- Save the workflow as a team skill (inputs / outputs

Skills Used

`$dynamic-tables`

4 demos • Builds the foundation for all subsequent vignettes

Demo 1.1 — Setup: Explore the source data

Demo 1.2 — Compare SAP and Oracle invoice schemas

Demo 1.3 — Generate Silver Dynamic Table

Demo 1.4 — Use the $dynamic-tables skill

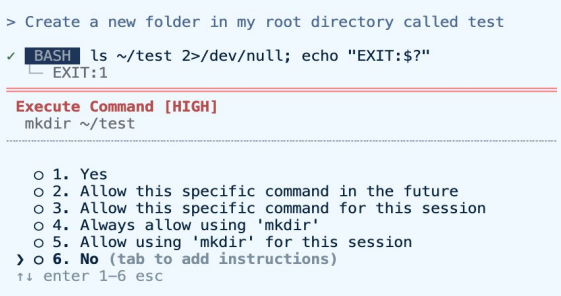
Demo 1.5 — Save one proof query

Execution Modes in CLI

A look back at how we can run our different permissions:

Confirm Mode (default)

Prompts the user before file edits, bash commands, etc. All proposed operations will classify risk level to user for approval.

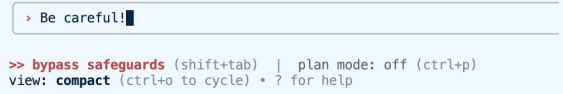


Plan Mode (/plan)

Agent polls user for additional detail and proposes a plan but doesn't execute any action until approved.

⚠️ Bypass Mode (/bypass)

Auto-approves all tool without polling user (not recommended - can be disabled via managed settings)



Bundled Skills in Snowflake

Pipeline Maintainer

Diagnose a stale pipeline, fix it, and onboard new sources

The Story

It's a month after launch. The Silver pipeline from Vignette 1 is live, but something's wrong — data looks stale and stakeholders are asking questions.

Meanwhile, the Finance Transformation PMO just sent over an Excel file: they want to onboard two more ERP systems (Baan from EMEA and Workday from Americas). The spreadsheet has column mappings, business rules, and open questions.

You need to diagnose the issue, translate those business requirements into technical specs, evolve the pipeline to handle 4 sources, and set up monitoring so this doesn't happen again.

What You'll Learn

- Turn an Excel requirements doc into executable change steps
- Capture the workflow as a reusable team skill/runbook
- Run our local skill
- Save materials to share updates with team members

Skills Used

`$skill-development`

5 demos • Requires Vignette 1 completed • Uses local (or staged) Excel file

Demo 2.1 — Read the local PRD

Demo 2.2 — Scaffolding a Custom Skill

Demo 2.3 — Run the PRD Evaluator Skill

Demo 2.4 — Apply the changes

Demo 2.5 — (optional) Save handoff materials

Build First Agent

Enable business users to ask natural language questions on our data

The Story

The AP consolidation pipeline is running smoothly with 4 source systems. But only engineers can query it — business users can't write SQL.

You'll create a semantic view that translates the Silver table into business-friendly dimensions and measures, then wrap it in a Cortex Agent that answers natural language questions about AP invoices.

But an agent is only as good as its answers. You'll build an evaluation framework — test questions, logged responses, and human feedback — then use that data to audit and improve the agent's semantic view iteratively.

What You'll Learn

- Generate a semantic view from live table metadata
- Create an agent with verifiable responses (answer + SQL + assumptions)
- Run evaluation on stakeholder created first golden dataset
- Optimize Data Agent to learn from wrong responses

Skills Used

`$semantic-view` `$cortex-agent`

4 demos • Requires Vignettes 1-2 completed • Introduces AI Layer

Demo 3.1 - Create Semantic View on Silver Data

Demo 3.2 - Create a Cortex Agent grounded on the semantic view

Demo 3.3 - Evaluate against a business user curated dataset

Demo 3.4 - Iterate on Agent from evaluation feedback

Core Capabilities in $Semantic-View-Optimize

CREATE
Create new semantic views from scratch

AUDIT
Audit semantic view, VQR testing, best practices, evaluation

DEBUG
Analyzing errors, fixing failures, root cause analysis for specific SQL queries

My query keeps failing	Uses Debug Mode
Create a semantic view for my sales table	Uses Creation Mode
Test all my VQRs still work after schema changes	Uses Audit Mode VQR testing
Why does “revenue by region” generate the wrong SQL	Uses Debug Mode
Add metrics and filters to improve query success	Uses Audit Mode

Core Capabilities in $Agent-Optimize

CREATE
Building new Cortex Agents

ADHOC TEST
Test questions, and build evaluation datasets

DEBUG QUERY
Debug specific query failures

OPTIMIZE
Improve agent performance for production

Why does “show me last quarter’s revenue” fail	Uses Debug Mode
Build customer support agent to query data + respond	Uses Creation Mode
I want to test on 20 common questions	Uses Audit Mode
Why does “revenue by region” generate the wrong SQL	Uses Debug Mode
Agent only works 60% accuracy on my test set	Uses Optimize

APPENDIX

How do we evaluate an agent?

Our bundled Agent Optimization Skill supports 4-evaluation approaches:

Ad-Hoc Testing

Interactively ask questions and manually review responses to explore agent behavior

Curated Dataset Testing

Run business stakeholder provider questions with expected answers; score with LLM judge

Feedback Based Testing

Review production queries in a Streamlit application and annotate responses

Evaluation Dataset Curation

Build a representative question set from production logs or create from scratch

This demo uses approach #2 : Curated Dataset Testing

- Business stakeholders provided 15-20 realistic questions with expected answers
- This collaboration is a common starting point for teams building out with Agents

Settings Deep Dive

Two Layers of Control

User settings:
`~/.snowflake/cortex/settings.json`
(controlled by individual user)

Managed settings:
`managed-settings.json`
(controlled by IT admin via MDM/SCCM)

Managed settings override user settings. Users cannot bypass admin policy.

Settings precedence (highest → lowest)

1. Managed settings (admin policy)
2. Session commands (/plan, /bypass)
3. CLI arguments
4. Environment variables
5. User config files
6. Defaults

What Admins Can Enforce

Permissions: Tool allowlists/denylists, disable bypass, restrict Snowflake accounts

Settings: Force no history, force sandbox enabled

Required: Enforce minimum CLI version

Defaults: Default connection, profile, theme

UI: Managed banner, hide dangerous options

Hooks Deep Dive

What it does & how it works

What it does: Custom pre/post-execution rules that auto-approve or block specific operations.

How it works:

- `hooks.json` defines rules triggered before or after tool calls
- Matcher targets a specific tool (Bash, Write, SQL, etc.)
- Hook script returns allow, deny, or a custom reason
- Hooks are not bypassed even in bypass mode

Customer takeaway: *"Your security team writes the rules. CoCo enforces them automatically."*

Example: Auto-approve safe bash, block rm -rf:

```
{
    "hooks": {
      "PreToolUse": [
        {
          "matcher": "Bash",
          "hooks": [{
            "type": "command",
            "command": "bash hooks/policy.sh"
          }]
        }
      ]
    }
  }
```

Hook script can inspect the command and return:

- `"permissionDecision": "allow"` → **approved**
- `"permissionDecision": "deny"` → **blocked with reason**

Connections Deep Dive

What it does & how it works

What it does: Named connection profiles scoping each session to a specific Snowflake environment, role, and warehouse.

How it works:

- Defined in `~/.snowflake/connections.toml` (shared with Snowflake CLI)
- Each profile sets account, role, warehouse, database, schema
- Admins can restrict which accounts are connectable via managed settings

Customer takeaway: *"Users get the right access for the right environment. No accidental prod writes from a dev session."*

Example: Separate dev vs. prod profiles:

```
[dev]
account = "myco-dev"
role = "DEVELOPER"
warehouse = "DEV_WH"
database = "DEV_DB"

[prod_readonly]
account = "myco-prod"
role = "ANALYST"
warehouse = "REPORTING_WH"
database = "PROD_DB"
```

Launch with: `cortex -c prod_readonly`

Governance - Roadmap

What we're building

AUDIT LOGGING & OBSERVABILITY DATA

Prompt and response logging for analysis of usage patterns and performance insights

COST CONTROLS & BUDGETS

Manage usage costs and prevent runaway costs on a user or account level

SUPPORT FOR CORTEX GUARDRAILS

Protection against prompt injection and other LLM risks. Planned for Q2.

CORTEX CODE PROFILES

Subsets of global configuration settings for skills, MCP servers, settings, hooks etc.
Select a profile at session start to determine behavior for that session